多源视频同帧对照处理系统

基于 Zynq-7020 的全硬件视频通路与可观测性设计

摘要

本作品在 Zynq-7020 上实现一条从千兆网口到 HDMI 的视频处理通路。上位机将 512×300、RGB565 格式的视频帧拆分为不超过 1392 字节的 UDP 报文，发送至 5001 端口；PL 从 RGMII 接口接收数据，经 MAC、UDP 过滤和按帧内偏移重组后写入 DDR 乒乓缓存。显示侧读回整帧，完成缩放、任意角度旋转、双线性插值和六级效果处理，与 OSD 叠加后以 1024×600、59.5 Hz 输出 HDMI 信号。像素数据不经过 PS，PS 仅负责命令解析、状态回读和 SD 卡回放。屏幕中央的竖缝左侧显示原图，右侧显示同一帧的处理结果，两侧共用同一套几何运算和固定 15 拍效果链，便于逐像素比较处理前后的差异。系统提供帧提交条件、坏包计数、处理窗完整性、链路帧间隔和时延等状态量，可通过串口、UDP 遥测和 AXI GPIO 读取。板上实测 8 秒接收 241 帧，帧率 30.01 fps；三阶段链路自检结果为 22.2/0/18.9 fps；QSPI 启动、串口、网络和 SD 回放均通过。最终实现 WNS 1.368 ns、WHS 0.048 ns，失败端点 0，LUT 28.13%、BRAM 70.00%。

第一部分　作品概述

选题背景

视频处理系统可以采用两种实现方式：一种将算法放在 PS 或 PC 上，PL 仅负责采集和显示，开发较快，但像素数据需要经过处理器；另一种将完整通路放在 PL 内，延迟确定，但调试信息较少。本作品采用第二种方式，在 Zynq-7020 的 PL 内实现从 RGMII 到 HDMI 的视频通路，并将帧提交条件、坏包计数、处理窗完整性和链路状态做成可回读的计数器。这样，算法是否生效、链路是否丢帧以及优化是否改变行为，都可以通过同一帧和同一组状态量进行判断。

功能与特性

本作品接收 PC 通过千兆网口发送的 512×300、30 fps、RGB565 视频，在 PL 内完成 RGMII 采样、MAC、UDP 过滤、按帧内偏移重组、DDR 乒乓缓存、缩放、任意角度旋转、双线性插值、六级效果处理、同帧左右对照、五行 OSD 和 1024×600 HDMI 输出。系统支持网络流、SD 卡回放和片内测试图三种片源，支持串口命令、UDP 遥测和 AXI GPIO 回读，并支持三阶段故障注入自检和 QSPI 断电启动。像素数据不经过 PS，效果链延迟固定为 15 个像素时钟。

应用领域

本作品适用于需要远程视频传输和状态监测的场景。一根网线可同时传输视频、链路状态和 PL 内部时延，现场不需要额外采集设备。在广播、安防和工业视觉中，系统可在线显示接收帧数、坏包数、处理窗完整性和显示延迟，便于判断故障来自链路还是处理逻辑。同帧左右对照可用于算法前后效果比较，帮助区分算法未生效和链路丢帧。当前设计面向 512×300 分辨率和低带宽链路，不涉及 1080p 或多路压缩视频。

主要技术特点

（1）像素通路全部在 PL 内实现，包括 RGMII 收包、帧重组、DDR 乒乓缓存、几何变换、效果处理、OSD 和 TMDS 输出，PS 仅负责控制与回读。（2）

一帧数据必须同时满足整帧字节到齐、所有源行被覆盖、本帧无坏包三个条件才会提交，避免半帧显示。（3）效果链的延迟固定为 15 拍，开启或关闭各级效果时链路延迟不变，便于同帧比较。（4）原图和处理图共用一套几何运算和源坐标，竖缝只切换输出抽头。（5）串口、UDP 遥测和 AXI GPIO 读取同一组计数器，便于交叉核对。

主要性能指标

项目	读数
输入 / 输出	512×300 @ 30 fps / 1024×600 @ 59.5 Hz
时序	WNS 1.368/WHS 0.048；TNS=THS=0
资源	LUT 28.13%，FF 8.55%，BRAM 70.00%，DSP 8.64%
板级	241 帧 / 8.03 s = 30.01 fps；自检 22.2/0/18.9；QSPI 启动
时延	效果链恒 15 个像素时钟；PL 内部帧时延 2–16 ms

主要创新点

（1）同帧对照：竖缝两侧共用几何运算、行标签和 15 拍效果链，比较的是同一帧。（2）帧提交条件：字节数、行覆盖位图和坏包计数同时满足才提交，作废原因可解释。（3）状态可验证：处理窗完整性、链路帧间隔和故障注入均有明确判据，异常时计数会变化。（4）整机可复现：全 PL 通路、QSPI 启动和串口/UDP/GPIO 回读形成完整链路，断电后可自行启动。

设计流程

需求分析与指标确定 → PL/PS 功能划分 → RTL 与 PS 固件实现 → 仿真台架与对照测试 → 综合与实现 → JTAG 上板 → QSPI 固化 → 文档与提交包整理。每个阶段保留判定记录或实测日志，未测量项标注为 NOT_MEASURED。

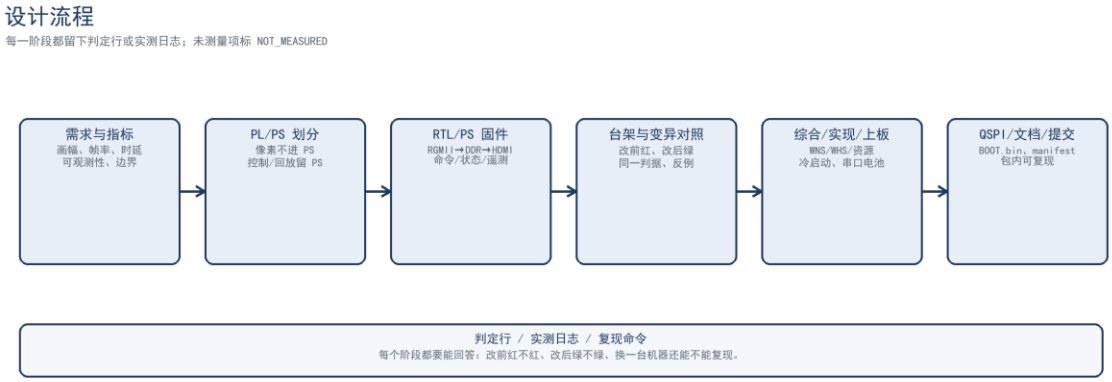

设计流程
每一阶段都留下判定行或实测日志；未测量项标 NOT_MEASURED
需求与指标
画幅、帧率、时延
可观测性、边界
PL/PS 划分
像素不进 PS
控制/回放留 PS
RTL/PS 固件
RGMII→DDR→HDMI
命令/状态/遥测
台架与变异对照
改前红、改后绿
同一判据、反例
综合/实现/上板
WNS/WHS/资源
冷启动、串口电池
QSPI/文档/提交
BOOT.bin、manifest
包内可复现
判定行 / 实测日志 / 复现命令
每个阶段都要能回答：改前红不红、改后绿不绿、换一台机器还能不能复现。

图 1　设计流程图

第二部分　系统组成及功能说明

整体介绍

系统由 PC 上位机、板载千兆 PHY、PL 视频通路、PS 控制与回放、HDMI 显示面板五部分组成。PC 将片源缩放到 512×300 并封装为 UDP 报文；PL 从 RGMII 接口接收数据，完成 MAC、UDP 过滤和按帧内偏移重组，只有整帧收满且行覆盖完整时才写入 DDR 乒乓缓存。显示侧读回整帧，经过几何变换、双线性插值、六级效果和竖缝选择，叠加 OSD 后以 1024×600 输出 HDMI。PS 通过 AXI GPIO 写控制字、读取状态，并负责串口命令、SD 卡回放和温度采样；UDP 遥测将同一组 PL 计数发送给 PC。像素数据不经过 PS，控制与数据通路分离。

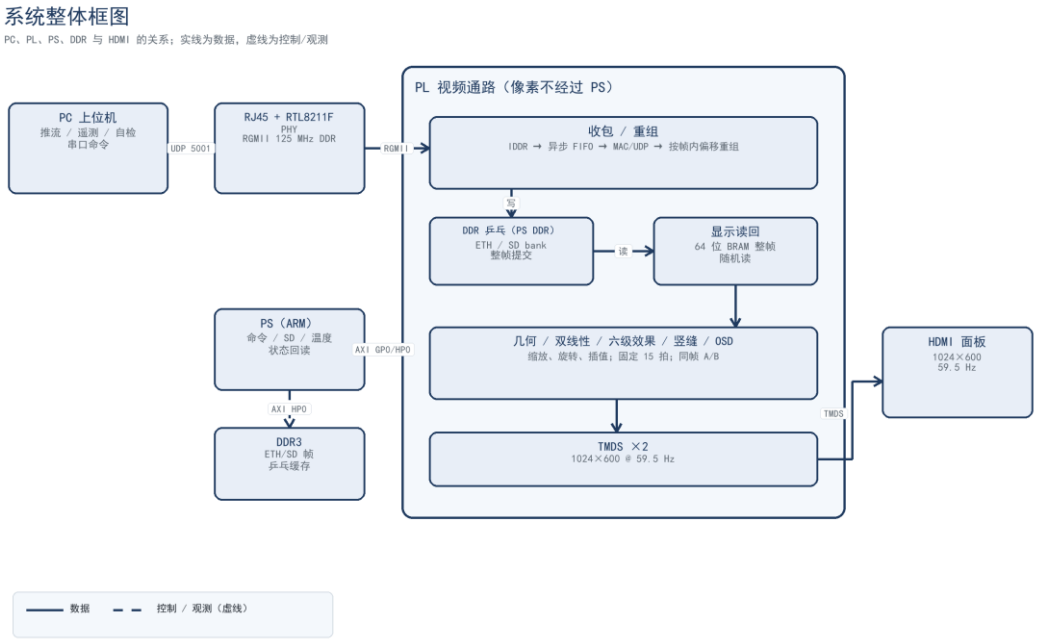


图 2　系统整体框图

硬件系统介绍

2.2.1 硬件整体介绍

硬件核心为 RK-ZYNQ7020-F 开发板，器件型号 xc7z020clg484-2，综合、实现和平台构建使用 Vivado/Vitis 2025.2.1。板载 DDR3、QSPI、microSD、USB-

C JTAG/UART、50 MHz 时钟、PL 侧 RJ45（RTL8211F-CG）和 HDMI 接口。PL 负责视频与网络数据通路，PS 负责命令、SD 卡、温度和状态回读，两者通过 AXI GP0/HP0 连接。

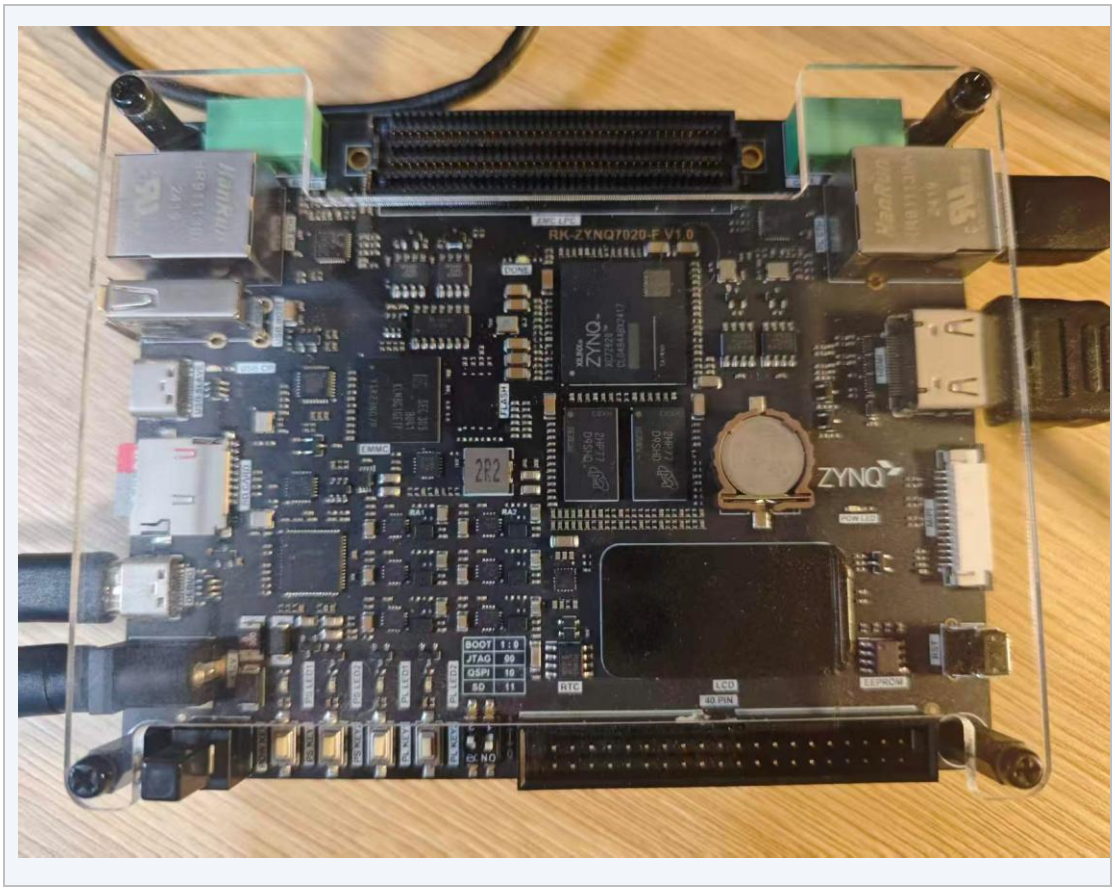


图 3　硬件整体照片

2.2.2 机械设计介绍

本作品未设计专用机械结构，实物由开发板、HDMI 面板、支架和线缆组成。演示时按屏幕在上、板卡在下的方式摆放。若后续增加外壳，可在此补充 CAD 图和实物照片。

2.2.3 电路各模块介绍

硬件采用现成开发板，未设计自研 PCB 和原理图。电路部分主要说明板级接口、时钟和关键信号：RGMII 在 PL 侧 RJ45 上以 125 MHz DDR 传输 4 位数据和控制信号；HDMI 输出 1024×600 TMDS 信号；QSPI、SD、UART/JTAG 连接 PS；50 MHz 板载时钟进入 MMCM，产生像素时钟、以太网时钟和串行时钟。

接口	关键信号	方向	说明
RGMII	rxd[3:0]、rx_ctl、rx_clk、txd[3:0]、tx_ctl、tx_clk	PHY ↔ PL	125 MHz DDR，千兆
HDMI	tmds_clk_p/n、tmds_data_p/n[2:0]	PL → 面板	1024×600 ， 59.5 Hz
QSPI	4-bit data/clk/cs	PS ↔ NOR	BOOT.bin 启动
microSD	4-bit data/cmd/clk	PS ↔ 卡	帧序列回放
USB-C	UART/JTAG	PC ↔ PS	115200 8N1 / 调试
时钟	sys_clk 50 MHz	板 → PL	MMCM 输入

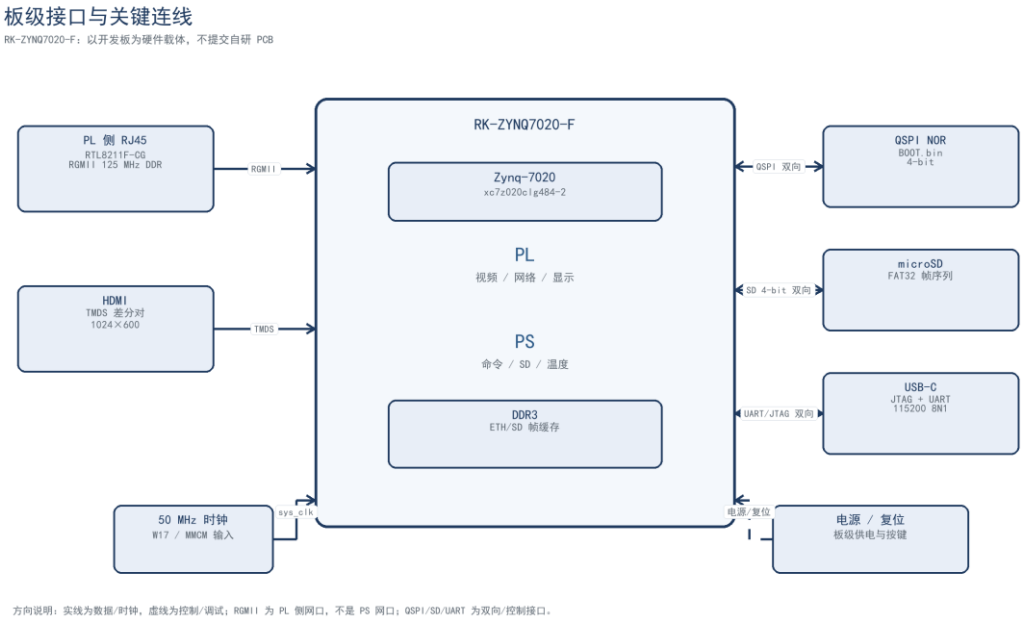


图 4 板级接口与关键连线

软件系统介绍

2.3.1 软件整体介绍

软件分为 PC 上位机、PS 裸机程序和 PL 逻辑三层。PC 侧使用 Python 和 Node 工具完成推流、遥测、链路自检和串口测试；PS 固件负责命令解析、控制字写入、状态读取、SD 卡回放和温度采样；PL 逻辑负责收包、帧重组、DDR 写入、显示、效果处理、OSD 和链路监测。控制通路使用 AXI GPIO，数据通路使用 AXI HP0，状态可通过串口、UDP 和 GPIO 读取。开发过程中使用大模型

辅助代码检索、测试脚本编写和文档整理，关键结论均通过仿真、实现报告或板上日志复核，协作记录见附录 A。

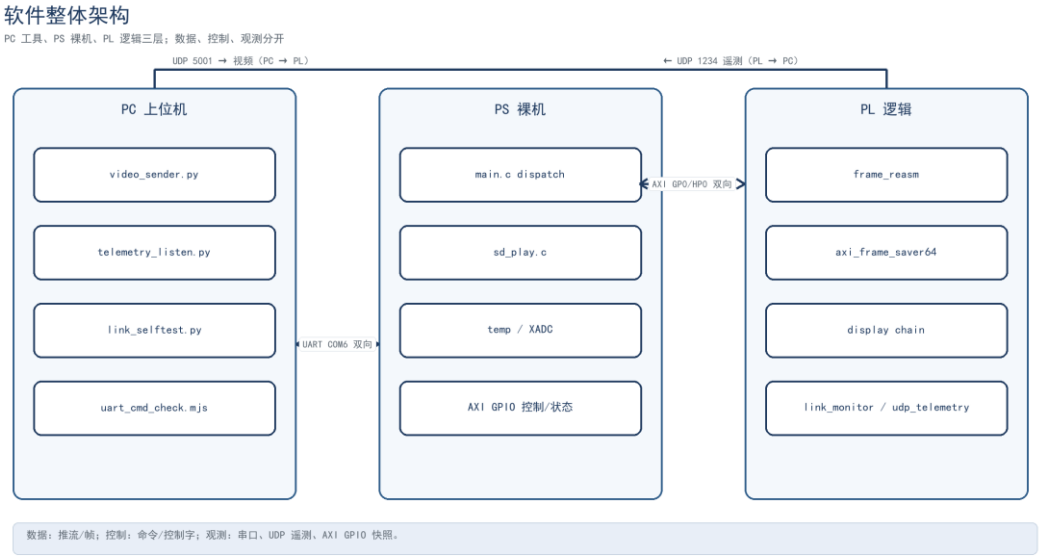


图 5　软件整体架构

2.3.2 软件各模块介绍

PS 命令层从串口逐行接收字节，按 CR/LF 分行，每行最多解析 8 个参数，再进入 dispatch 分派。每条命令只有执行、拒绝和无法识别三种结果。图 6 给出从 UART 接收到控制字写入或状态回读的流程，主要状态变量包括 cur_thr、cur_sel、cur_split、cur_gamma 和 cur_osd。

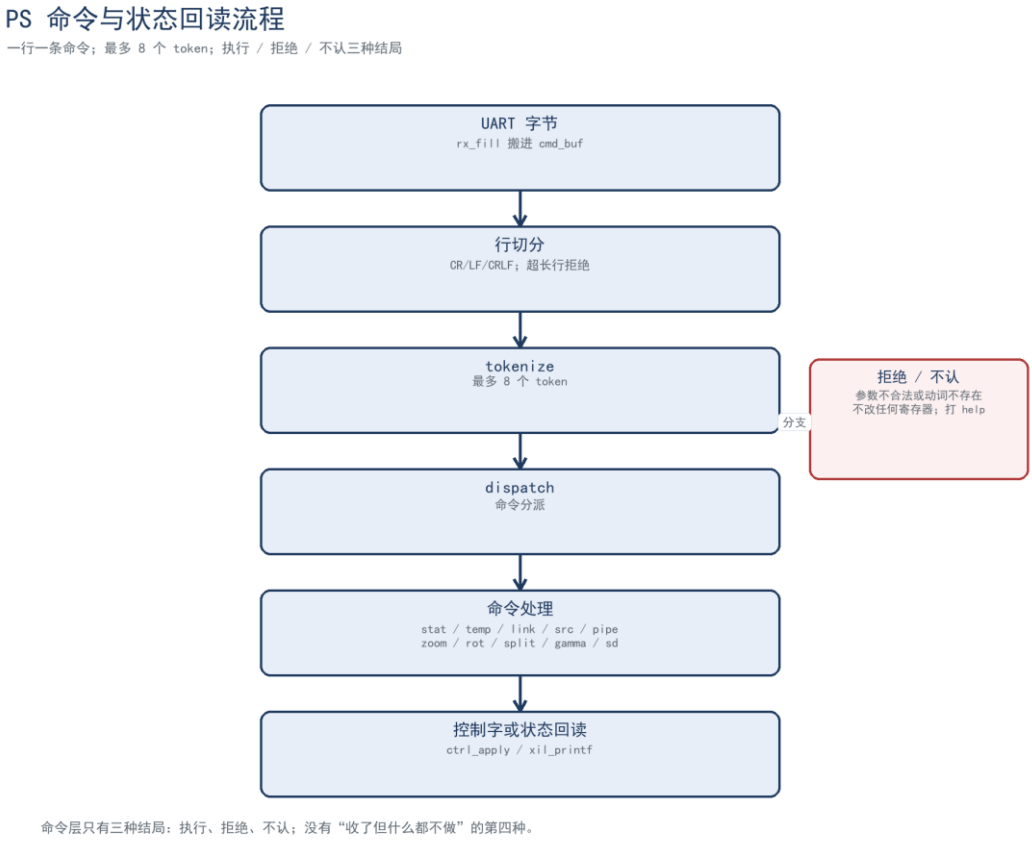


图 6　PS 命令与状态回读流程

视频侧由 PC 推流、PL 收包与重组、DDR 写入、显示链读回四部分组成。主要状态量包括包偏移、载荷长度、字节计数、行覆盖位图、p_good、64 位打包字数、DDR 乒乓 bank 和提交脉冲。

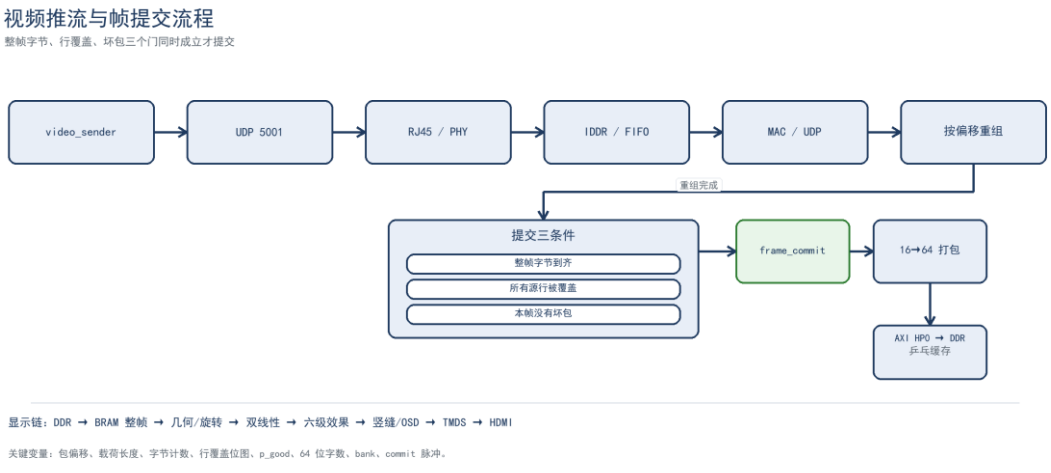


图 7　视频推流与帧提交流程

模块	主要输入	主要输出	关键变量
main.c dispatch	UART 行、当前控制状态	控制字、状态行	cur_thr、cur_sel、cur_split、cur_gamma
sd_play.c	SD 文件/帧号	DDR 帧、播放状态	frame index、playing
video_sender.py	视频帧/内置图	UDP 报文	offset、payload、fps
telemetry_listen.py	UDP 1234 信标	帧数/坏包/时延/pane	frames、bad、aborts、lat_ms、pane_bad
link_selftest.py	clean/fault/clean	PASS/FAIL	fps delta、abort rate
frame_reasm	UDP 包	frame_commit	bytes、row bitmap、p_good
axi_frame_saver64	16 位像素	AXI 突发	64-bit words、FIFO、tail
link_monitor	帧/包事件	链路计数	gap_last/min/max/sum、stall、live

2.4 软硬件划分依据与接口设计

软硬件划分遵循三条原则：像素数据不进入 PS；PS 只负责低频控制、状态回读和 SD 卡回放；跨时钟域控制使用 AXI GPIO 准静态总线，视频数据使用 AXI HP0。主要依据是带宽和确定性：512×300×16 bit×30 fps 约为 73.7 Mbit/s，整帧 307 200 字节；如果每个像素都经 PS 搬运，CPU 和 DDR 带宽会被像素流占用，中断抖动也会影响显示节拍。将像素通路留在 PL 后，PS 只处理命令和遥测，控制与数据通路分离。

接口	方向	时钟 / 速率	用途
RGMII	PHY ↔ PL	eth_rxc 125 MHz DDR	视频收包
AXI HP0	PL → DDR	clk_fpga_0	整帧写回 / 读回
AXI GP0	PS ↔ PL	clk_fpga_0	控制字、状态 lane
UART	PS ↔ PC	115200 8N1	命令与回读
UDP 5001 / 1234	PL ↔ PC	eth_rxc	视频流 / 1 Hz 遥测
HDMI	PL → 面板	1024×600 @ 59.5 Hz	TMDS 输出
microSD	PS ↔ 卡	4-bit SD	帧序列回放

控制字、遥测字段和串口命令在 report/interface-table.md 与 report/commands.md 中列出。跨时钟控制采用准静态快照，并使用 set_max_delay -datapath_only 10 ns 约束，避免将低频控制路径按高频数据路径处理；RGMII 发送侧增加 ODDR 输出窗，接收侧使用 IDDR 和时钟组约束。

2.5 时序与资源优化

时序优化以不改变板级行为为前提，目标是提高最差负裕量并减小复位网络的扇出。交付基线的全局 WNS 为 1.341 ns、WHS 为 0.025 ns，最差路径位于 sys_clk 到 clkout3 的复位恢复，以及 link_monitor 中由单个 LUT 驱动 539 个端点的复位分发树。首先尝试将 eth_rst_async 打一拍，全局 WNS 降至 1.277 ns；随后尝试将协议域异步复位改为同步复位，虽然减少了异步复位端点，但全局 WNS 降至 0.674 ns；使用 pblock 固定复位锚点的方案对结果没有影响。最终采用复位分发树拆分：将 rst_lm_sync2 的第三级复制为三个同沿副本，

link_monitor 的三组复位分别使用一个副本。

方案	WNS(ns)	WHS(ns)	失败端点	主要改动	结果
交付基线	1.341	0.025	0	—	基线
方案一：复位打一拍	1.277	0.051	0	将 eth_rst_async 打一拍	未采用，全局余量下降
方案一+方案二	1.139	0.048	0	两种改动叠加	未采用，布局重排抵消收益
方案三：同步复位	0.674	—	0	协议域异步复位改同步	未采用，短板转移
方案四：pblock	1.368	0.048	0	固定复位锚点	未采用，结果不变
方案二：复位分发树拆分	1.368	0.048	0	复位分发树拆为三个副本	采用

资源 / 功耗	交付基线	最终版	变化
LUT	14 967（28.13%）	14 965（28.13%）	−2
FF	9 095（8.55%）	9 097（8.55%）	+2
BRAM	98（70.00%）	98（70.00%）	0
DSP	19（8.64%）	19（8.64%）	0
功耗（估算）	2.406 W	2.407 W	+0.001 W

最终方案的收益主要来自结构改善：复位分发由 539 个端点的单点驱动变为约 3×180 个端点。八次构建的全局 WNS 在 0.674–1.368 ns 之间，1.368 ns 已接近当前异步复位结构和双 MMCM 时钟架构的余量上限。优化前后行为一致：19 支标准回归 fail=0，完整视频台架判定结果不变，效果链仍为固定 15 拍。最终版位流经 JTAG 下载后全项通过，连续流 30.01 fps，自检 22.2/0/18.9；QSPI 启动沿用交付基线镜像的验收结果（29.95 fps、22.0/0/18.9），最终版的 QSPI 固化脚本保留在 board/scripts/make_boot_image.sh。

第三部分　完成情况及性能参数

整体介绍

实物由开发板、HDMI 面板、PC、网线、USB-C 线和电源组成。板卡 PL 侧 RJ45 连接 PC，HDMI 连接 1024×600 面板，USB-C 提供 JTAG 和 UART。演示时屏幕在上、板卡在下，线缆从侧面走线，避免遮挡 OSD。

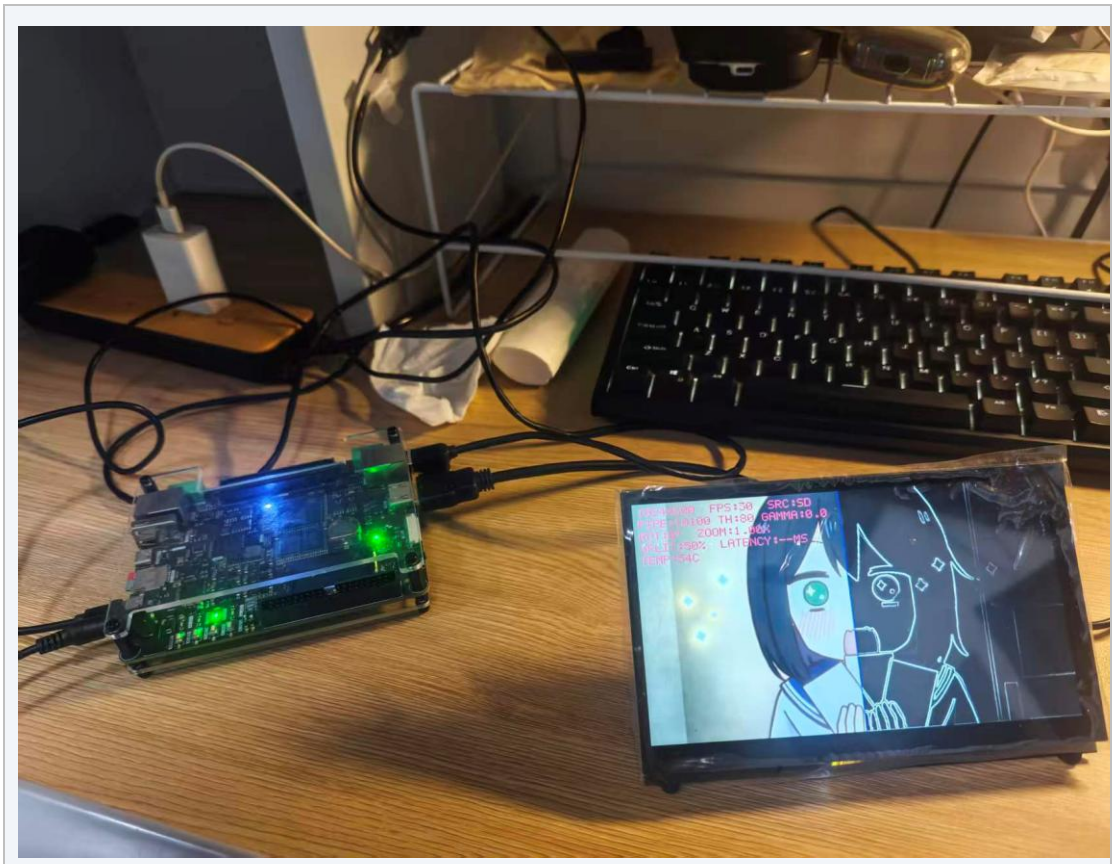


图 8　系统实物正面

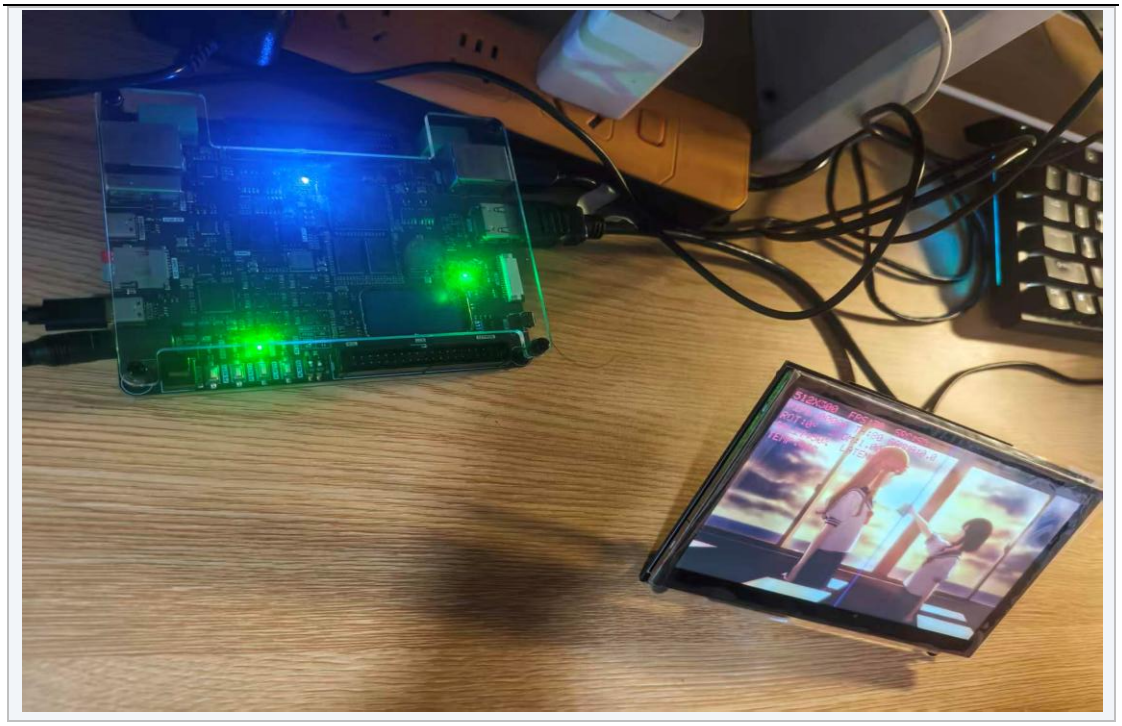


图 9　系统实物斜 45°

工程成果

3.2.1 机械成果

本项目没有自研机械件，实物布置、支架和线缆走向见图 8 和图 9。若后续增加外壳，可补充装配图和实物照片。

3.2.2 电路成果

使用 RK-ZYNQ7020-F 开发板，没有自研 PCB。板级接口、关键信号和连线见图 4，板卡整体照片见图 3。

3.2.3 软件成果

软件成果包括串口命令与状态回读、屏幕 OSD 和 PC 遥测界面。图 10 将三部分放在同一画面中：左侧为串口 stat/link 输出，中间为屏幕 OSD，右侧为 telemetry_listen.py 的遥测结果。



图 10　软件界面与 OSD

特性成果

表 3 列出可由提交包内日志复核的读数，原始日志位于 board/measured、build/reports 和 sim/results。

特性	判据	读数
帧提交	整帧字节 + 行覆盖 + 无坏包	241 帧 / 8.03 s = 30.01 fps
处理窗完整性	185°、split video、灰度单级	pane_bad = 0，计数有效
故障注入	干净 → 每 100 包丢 1 包 → 干净	gap_valid=1； gap_max=49 ms
帧间隔	link clr → 推流 → 停流 → 重启	DONE/EOS，ping 5/5（交付基线镜像）
QSPI 冷启动	断电重上、串口、ping	DONE/EOS，ping 5/5
时序	实现后时序报告	WNS 1.368 ns，WHS 0.048 ns，失败端点 0
资源	实现后资源报告	LUT 28.13%，BRAM 70.00%，DSP 8.64%
长时与插拔	连续挂机数日；ETH/SD 插拔	未见异常；E1–E5 已确认

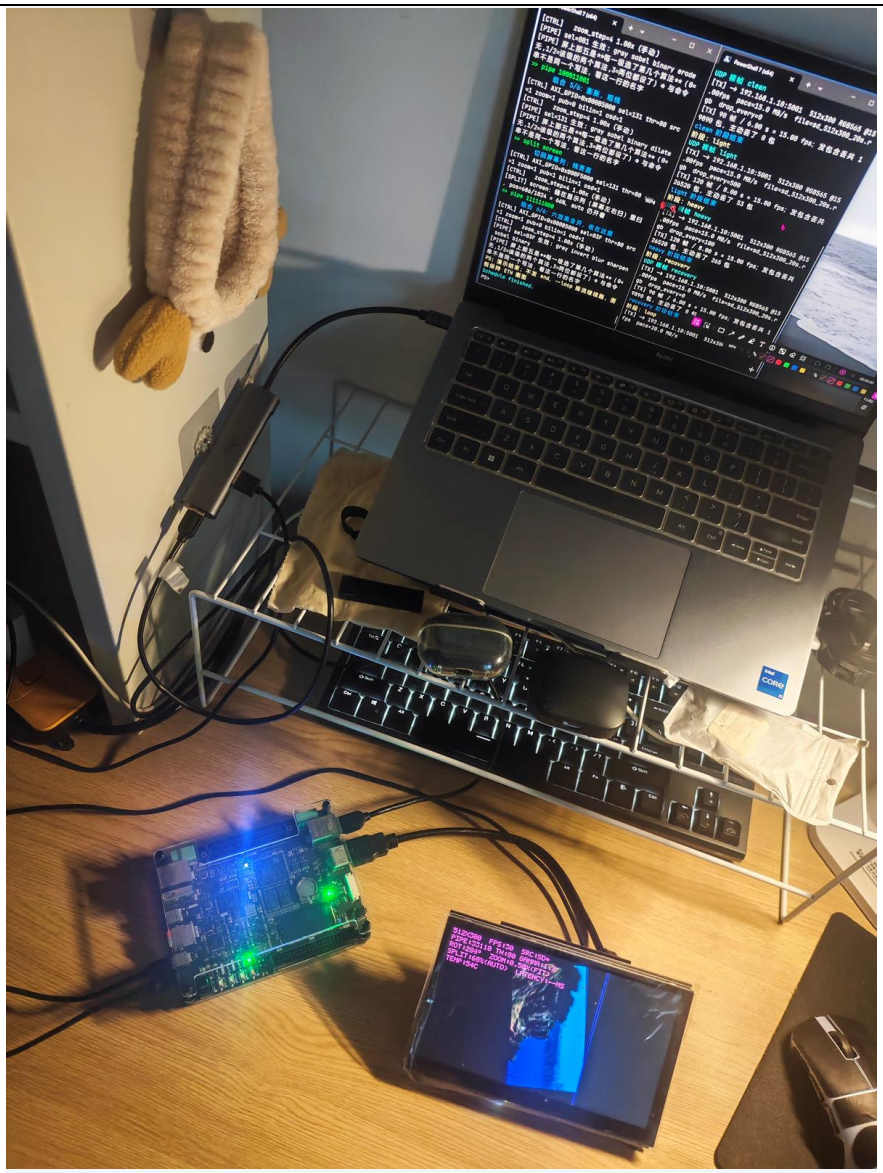


图 11　系统整体实物

第四部分　总结

可扩展之处

当前分辨率为 512×300，BRAM 占用 70%，主要由整帧缓存决定。提高分辨率需要改用 DDR 瓦片缓存或分块几何处理；1080p60 原始视频超出单根千兆网口的带宽，需要压缩或升级链路。RGMII 发送侧所在 HR bank 没有 ODELAYE2，仍有约 0.845 ns 的 TskewT 保持缺口，产品化时需要更换 bank 或增加 ODELAY，并进行眼图测试。后续还可增加远程升级、双分区、设备发现和长期可靠性测试。

心得体会

调试过程中感受最深的是，画面正常并不等于数据完整。最初主要关注帧率和显示效果，后来在丢包测试中发现，字节数足够时仍可能有部分行没有写入，屏幕会出现半帧。将整帧字节数、行覆盖位图和坏包计数分开统计后，才能判断一帧是否应该提交，显示侧也有了明确的切换时机。

同帧对照的实现也经过调整。早期原图和处理图各走一条缩放链，即使参数相同，竖缝两侧也会差一行。后来改为共用一套几何运算和源坐标，竖缝只切换输出抽头，并将效果链固定为 15 拍，两侧才能对应到同一像素。固定延迟的目的是把几何差异和算法差异分开，便于观察处理效果。

时序优化时，我们先阅读实现报告的逐时钟余量和路径分解，再决定改动。复位分发树的扇出较大，拆分后 WNS 和 WHS 都有改善；给复位信号打一拍、将异步复位改为同步复位、使用 pblock 等方案也做过实验，但因全局余量下降或没有效果而未采用。这个过程说明，局部路径改善不一定带来全局收益，布局变化可能抵消逻辑上的优化。

上板调试中，JTAG 能运行并不代表交付可用。我们补充了 QSPI 启动、断电重上、串口命令测试和长时间运行，也遇到过 bootgen 返回成功但镜像未更新、命令函数存在但未接入分派等问题。现在交付包将这些检查做成脚本和记录，便于复现和复核。

如果继续改进，我会在编写数据通路之前先确定状态量和提交条件，再实现显示与控制逻辑。这样后期调试可以直接查看计数器，而不必反复猜测问题位置。

第五部分　参考文献

[1] AMD/Xilinx. Zynq-7000 SoC Technical Reference Manual (UG585).

[2] AMD/Xilinx. 7 Series FPGAs Clocking Resources User Guide (UG472).

[3] AMD/Xilinx. UltraFast Design Methodology Guide (UG949).

[4] AMD/Xilinx. Vivado Design Suite User Guide: Synthesis (UG901).

[5] AMD/Xilinx. Vivado Design Suite User Guide: Implementation (UG904).

[6] AMD/Xilinx. Zynq-7000 Software Developers Guide (UG821).

[7] HDMI Licensing, LLC. HDMI Specification 1.4b.

[8] IEEE. IEEE Standard for Ethernet, 802.3-2018.

[9] IETF. RFC 768: User Datagram Protocol.

[10] Alex Forencich. verilog-ethernet: Verilog Ethernet components (GitHub repository), used as a comparison reference.

附录 A 大模型协作记录

开发过程中使用大模型辅助代码检索、测试脚本编写、报告整理和问题定位。协作方式为：由人确定目标、架构和验收标准，主 agent 负责修改代码、运行检查和整理文档，只读子 agent 负责查找文件和核对报告。子 agent 的结论需要回到源文件、日志或判定行复核，不能仅凭文字说明认定完成。完整的会话记录、提示词和纠错轨迹位于 report/collaboration/；原始导出包含本机路径，不随提交包提供。

角色	负责	边界
人	确定目标与架构、判断画面、上板验收、决定提交范围	人眼验收与平台字段由人确认
主 agent	阅读跨文件关系、修改 RTL/PS/台架、运行检查、整理文档	修改必须有判据和回归
只读子 agent	查找文件、检查调用、审阅文档、比对报告	不修改文件，结论回主 agent 复核

以下三条纠错记录可以复核。第一，帧提交条件：模型最初只列出字节数和坏包，遗漏了行覆盖；使用“字节数足够但行不完整”的激励后问题复现，随后补充了行覆盖位图条件。第二，时序优化：模型曾建议通过实现策略将负 slack 转移到其他路径，该建议未被采纳；按报告逐时钟分析最差路径后，最终采用复位分发树拆分，并保留其他方案的失败读数。第三，link 命令：函数已经实现但未接入 dispatch，板上执行时返回无法识别；接入分派和帮助信息后，105 条串口命令测试通过。

模型回答摘录（要点，非逐字原文）：第一次询问帧提交条件时，模型回答“收满 307200 字节且坏包计数为 0 即可提交”，没有列出行覆盖位图；用“字节数够、只覆盖 3/4 行”的激励复现后，回答补充为“整帧字节数、行覆盖位图、坏包门三者同时成立”。询问时序优化时，模型回答“可用实现策略把负 slack 转移到其他路径”；按逐时钟报告复核后发现只是转移了失败端点，未采纳。询问 link 命令时，模型回答“函数已实现，命令应可用”；板上回读显示 dispatch 表没有该入口，接入后才通过。上述回答的要点与修正过程见 report/collaboration/sessions/ 和 corrections.md。

实际使用过的提示词示例如下（原文摘录）：

```
只读 src/rtl/eth/frame_reasm.v，列出提交一帧所需的全部条件。
每条条件说明由哪个信号、哪个 always 块维护，不要推测。
给出一个能使“字节数足够但行不完整”复现的台架激励。
读取 build/reports/build_timing_summary.rpt，按时钟列出 setup/hold 最差路径。
对每条负 slack 给出起点、终点、logic/route 分解和候选修改方案；
不要直接建议放宽约束。
```

提示词中读取的是优化前报告 build/reports/build_timing_summary.rpt；最终版读数取 build/reports/timing_summary.rpt，优化前基线保留在原文件，便于前后对照。

智能体工作流设计为五段：人确定目标与验收标准 → 主 agent 阅读代码、报告和日志并修改文件 → 只读子 agent 并行定位文件、检查调用和比对文档 → 台架、实现或上板复验 → 人签收画面、模板和平台字段。主 agent 拥有写权限，负责 RTL、PS 固件、脚本和文档；只读子 agent 不修改文件，结论必须回到主 agent 复核。每次修改先写能复现问题的判据，再改实现，最后用同一判据复跑；工具调用、命令输出和失败读数保存在 report/collaboration/、build/reports/、board/measured/ 和 sim/results/。

附录 B　技能包提炼

skill/ 目录整理了开发过程中可复用的经验，共 59 条，按用途分为八组。每条 SKILL.md 包含六节：用途、适用场景、不适用或失效条件、前置条件、使用方法、判读与失败分叉。另一支队伍可以根据症状索引直接查找条目，不需要通读整个目录。

组	条数	内容
prompts/	8	提示词工作流：软硬件划分、报告定位瓶颈、单轮修改、评审、只读审查
templates/	4	工程骨架、接口契约、报告表单、脚本模板
scripts/	5	黄金比对、指标采集、寄存器映射检查、复现检查、契约生成
pitfalls/	24	判据误判、CDC、约束覆盖、文档版本、构建竞态、工具探针陷阱

组	条数	内容
runtime/	8	DDR 带宽、原子窗口回读、UDP 偏移重组、板级以太网/串口等运行时事实
references/	7	WNS 逻辑/走线、台架语法子集、报告字段图、判定行格式
zynq-video-rtl-debug/	1	L0–L4 分层调试路径
frameworks/	2	Vivado/板卡侧与 PYNQ Overlay 两条通用框架技能

每条 SKILL.md 固定为六节：一句话用途、适用场景、不适用或失效条件、前置条件、使用方法、判读与失败分叉。赛题要求的适用范围、使用方法、失效条件和验证效果分别从第 2、5、3、6 节读取；条目来源写在出处段和 _meta/sources.md。

条目来源包括四类：实际故障（判定行未输出却报告通过、catch 保存返回码被误读、link 函数未接入分派）、重复出现的问题（Tcl 查询返回空集合被误认为对象不存在、报告字段随工具版本变化）、测量差异（综合后与布线后的 WNS 不一致）以及外部资料（AXI、RGMII、CDC 和公开以太网实现的取舍）。被否定的判断保留原记录、复现实验和最终结论。

验证方法为：在相同模型、上下文和工具版本下，分别在不使用技能包和使用技能包时运行同一任务，控制变量保持一致。确定性检查要求同一输入运行两次结果一致；非确定性检查要求已知错误输入能够复现问题、已知正确输入能够通过。平台边界在条目中注明：7 系列的 IDELAY/BUFIO/ODDR 用法不能直接用于 UltraScale+，跨板卡迁移需要重新验证；工具版本以 Vivado/Vitis 2025.2.1 为准，报告字段变化时需要重新确认解析位置。

附录 C 复现说明

复现分为五步：获取源码、构建位流、运行仿真、构建 PS 应用、上板或固化 QSPI。每一步给出前置条件、命令和期望结果。完整步骤、常见问题和环境变量说明见 report/reproduction.md。

项	要求
器件	xc7z020clg484-2，Zynq-7020 开发板
工具	Vivado / Vitis 2025.2.1；ModelSim（MSIM_BIN 指到安装目录）
主机	Windows 10/11 或 Linux；Python 3.10+；Node.js 18+（可选）

项	要求
网络	PC 192.168.1.100/24，板卡 192.168.1.10/24，直连千兆
串口	FT4232 B 通道，COM6，115200 8N1
显示	1024×600 HDMI 面板

构建与仿真：

```
vivado -mode batch -source build/build.tcl
MSIM_BIN=<ModelSim>/win64 bash build/sim/run_msim.sh rows udp_reasm \
    reasm_bounds udp_telemetry eth_ctrl_arb eth_top_rgmii split_ctrl \
    osd_overlay link_monitor
MSIM_BIN=<ModelSim>/win64 bash build/sim/run_msim.sh video_pipeline_top
```

PS 应用与上板：

```
PS_CC=<Vitis>/gnu/aarch32/nt/gcc-arm-none-eabi/bin/arm-none-eabi-gcc.exe \
PS_BSP=<平台构建生成的 BSP 目录> PS_OUT=board/project/ps_app.elf \
python build/ps/build_ps_app.py
VP_XSDB=<Vitis>/bin/xsdb.bat VP_VIVADO_BIN=<Vivado>/bin \
    bash board/scripts/board_flash.sh
python src/host/video_sender.py --demo --seconds 8
python src/host/telemetry_listen.py 13
python src/host/link_selftest.py 192.168.1.10 6
```

期望结果：构建结束时序报告失败端点为 0；每支台架末尾出现 RESULT ... PASS，完整视频台架专项通过；上板出现 FLASH: GREEN；8 秒推流约 241 帧（30 fps），13 秒收到 13 个遥测包；三阶段自检通过。QSPI 固化使用 board/scripts/make_boot_image.sh 和 board/tcl/flash_qspi.tcl，写入后断电重上，再检查串口、ping、stat、link 和遥测。JTAG 下载是易失的，断电后会回到 QSPI 镜像。